LORE-KV: Separating Log-Probability from Entropy

Reparametrizing Eq. 3 so the likelihood and entropy channels are independent. $\hat{D}_m$ **is retained unchanged** as a third channel, off by default at $\lambda_D = 0$, and tested for incremental value.

Notation. $h_u^{(m)} = (X, y_{<u}^{(m)})$;

$$c_u^{(m)} = \log p_\theta!\big(y_u^{(m)}\mid h_u^{(m)}\big) + H!\big(p_\theta(\cdot\mid h_u^{(m)})\big), \qquad \bar c_m = \tfrac1T!\sum_u c_u^{(m)}, \qquad \bar H_m = \tfrac1T!\sum_u H_u^{(m)},$$

$$V_u = \mathrm{Var}_{y\sim p_\theta(\cdot|h_u)}\big[\log p_\theta(y)\big]\ \text{(varentropy)}, \qquad \bar V_m = \tfrac{1}{T}\sum_u V_u^{(m)}, \qquad \tilde c_m = \bar c_m / \sqrt{\bar V_m / T}.$$

$\hat{D}_m$ is Eq. 5 exactly as written in the draft.

Part 1 — Paper changes

1.1 New equation before Eq. 3 (§2.3)

$$c_u^{(m)} = \log p_\theta\big(y_u^{(m)} \mid h_u^{(m)}\big) + H\big(p_\theta(\cdot \mid h_u^{(m)})\big)$$

Why: $\log p_\theta(y_u \mid h_u)$ confounds how uncertain the model was at h_u with how improbable the branch actually taken was, relative to that uncertainty. Only the second is evidence about the particle. Adding H_u back removes the first.

1.2 Replace Eq. 3

$$\boxed{R_m \;=\; \beta\,\bar c_m \;-\; \gamma\,\bar H_m \;-\; \lambda_D\,\hat D_m} \qquad \omega = \mathrm{softmax}(R) \in \Delta^{M-1}$$

Three independent channels: β prices **branch atypicality** (did this particle take steps the model itself deemed improbable at the contexts it visited); γ prices **context uncertainty** (an explicit, deliberate preference for low-entropy histories, matching the near-greedy deployment operating point §2.3 already argues for); λ_D prices **ensemble disagreement**, unchanged from Eq. 5.

Defaults: $\lambda_D = 0$. Optionally replace $\bar c_m$ with $\tilde c_m$ (§1.5); gate that choice on test V5.

1.3 Proposition 1 (nesting) — and the two false statements it corrects

Since $\frac{1}{T}\sum_u \log p_\theta(y_u \mid h_u) = \bar c_m - \bar H_m$,

$$R_m^{\text{old}} \;=\; \bar c_m \;-\; (1+\lambda_H)\,\bar H_m \;-\; \lambda_D \hat D_m,$$

so in $(\beta, \gamma, \lambda_D)$ coordinates:

Draft configuration	$(\beta, \gamma, \lambda_D)$
Uniform weights	$(0, 0, 0)$
"Log-probability only" ($\lambda_H{=}\lambda_D{=}0$)	$(1,\ \mathbf{1},\ 0)$
LORE-KV /E ($\lambda_H{=}0.3$)	$(1,\ 1.3,\ 0)$
LORE-KV /D ($\lambda_H{=}0, \lambda_D{=}0.3$)	$(1,\ \mathbf{1},\ 0.3)$

Correction 1 — delete from §2.3: *"setting both to zero leaves log-probability weighting alone."* False. $\lambda_H = \lambda_D = 0$ is $\gamma = 1$: a full unit of entropy penalty. The Table 4 row labelled "Log-probability only" is not a no-penalty control.

Correction 2 — delete from §2.3 and §3: any claim that `/E` and `/D` *"differ only in which penalty is active."* False in two ways. `/D` also carries $\gamma = 1$ — it does not replace entropy with disagreement, it **adds disagreement on top of a full entropy penalty**. And the pair $(1, 1.3, 0)$ vs $(1, 1, 0.3)$ differs in *both* γ and λ_D, so it is not a single-factor comparison.

This is the constructive point. The configuration that actually tests your hypothesis — disagreement *instead of* entropy — is $(\beta, \gamma, \lambda_D) = (1,\ 0,\ \lambda_D)$, and it was unreachable in the old parametrization. Reparametrizing is what makes the `/D` variant testable as intended.

Also add: the reported λ_H sweep varies $\gamma \in [1, 1.3]$ with β pinned at 1 — a 23% perturbation along one ray, never reaching $\gamma = 0$. This is why Table 4 shows 40.29 vs 40.28 and Table 11 shows 58.39 vs 58.43. Stated this way, the null result is explained rather than left implying the weighting apparatus is inert.

1.4 Proposition 2 (centering)

For $y_u \sim p_\theta(\cdot \mid h_u)$:

$$\mathbb{E}[c_u \mid h_u] = \sum_y p_\theta(y \mid h_u) \log p_\theta(y \mid h_u) + H_u = -H_u + H_u = 0,$$

hence $\mathbb{E}[\bar{c}_m] = 0$ and $S_t = \sum_{u \le t} c_u$ is a martingale w.r.t. the natural filtration. Holds per particle at every $M \ge 1$, with no reference to the ensemble — unlike $\hat{D}_m$, which is a sample-mean artefact and requires $M \ge 3$ (§1.10).

1.5 Proposition 3 (variance) and the optional normalization

For $u < v$, c_u is h_v-measurable, so $\mathbb{E}[c_u c_v] = \mathbb{E}[c_u\, \mathbb{E}[c_v \mid h_v]] = 0$. Therefore

$$\mathrm{Var}(\bar{c}_m) \;=\; \frac{1}{T^2} \sum_{u=1}^{T} \mathbb{E}\big[V_u\big].$$

Consequence: the across-particle spread of $\bar{c}_m$ scales with the prompt's varentropy, so a single global softmax temperature is not calibrated across prompts. Optionally use $\tilde{c}_m = \bar{c}_m / \sqrt{\bar{V}_m / T}$, which has unit scale by construction and is defined at every $M \ge 1$. **Report this as an ablated switch, not a default** — test V5 decides whether it earns a place.

1.6 Scope the "What the weighted ensemble estimates" paragraph (§2.3)

The current text claims the softmax gives a self-normalized importance-weighted estimator with target $\pi_\eta(Y) \propto p_\theta(Y \mid X)\exp(\eta R(Y))$. That claim requires R_m to be a function of $Y^{(m)}$ alone. Split it:

At $\lambda_D = 0$ (the default), R_m depends on $Y^{(m)}$ alone and Eq. 6 is self-normalized importance-weighted with target $\pi(Y) \propto p_\theta(Y \mid X)\exp(\beta\bar{c}(Y) - \gamma\bar{H}(Y))$. To first order, $\mathbb{E}_\pi[f] = \mathbb{E}_{p_\theta}[f] + \beta\,\mathrm{Cov}_{p_\theta}(f, \bar{c}) - \gamma\,\mathrm{Cov}_{p_\theta}(f, \bar{H}) + O(\|(\beta,\gamma)\|^2)$. Since $\mathbb{E}_{p_\theta}[\bar{c}] = 0$, the β channel contributes no zeroth-order shift; the mode-seeking tilt toward the deployment operating point is carried explicitly by γ.

At $\lambda_D > 0$, $\hat{D}_m$ depends on all M trajectories, so the weights are those of an interacting-particle estimator rather than importance weights for a fixed law. A fixed target is recovered asymptotically, since $D_m \to \mathbb{E}_{Y' \sim p_\theta}[d(Y, Y')]$ as $M \to \infty$, but not at $M = 4$.

Two things this fixes. It makes Contribution 3 ("gives the estimator an explicit target distribution") true as stated for the default method and honestly qualified for the disagreement branch. And it stops the paper from claiming, as §2.3 currently does, that $\hat{D}$ "leaves the ensemble's centre of mass in place" — a zero-mean statistic in the exponent does not give an unbiased tilt.

Do not overclaim centering either. $\mathbb{E}[\bar{c}] = 0$ does not imply $\mathrm{Cov}(f, e^{\beta\bar{c}}) = 0$. The guarantee is first-order.

1.7 Drop η — not identifiable

$\mathrm{softmax}(\eta[\beta\bar{c} - \gamma\bar{H} - \lambda_D\hat{D}]) = \mathrm{softmax}([\eta\beta]\bar{c} - [\eta\gamma]\bar{H} - [\eta\lambda_D]\hat{D})$: four symbols, three degrees of freedom. In the current parametrization η scales all channels together, so it moves along a ray and is not an independent knob either. Remove from Eq. 6, Table 6 and the config; absorb into $(\beta, \gamma, \lambda_D)$.

1.8 Fix Eq. 3 vs Algorithm 1 line 7

Eq. 3 uses the **mean** $\frac{1}{T}\sum_u \log p_\theta$; Algorithm 1 line 7 writes the **sequence total** $\log p_\theta(Y^{(m)})$. At $T{=}8$ that is an 8× difference in effective softmax temperature. Check which the code implements, make both the mean. Test V2 detects a mismatch as an exact T-fold gap.

1.9 State the sampling-law condition (§2.2 after Eq. 1; Appendix D.1)

Propositions 2 and 3 require the particles to be drawn from the law they are centered against. We sample at temperature 1 without truncation, so $q = p_\theta$ and $\mathbb{E}[c_u \mid h_u] = 0$ holds exactly. Under top-k/top-p /temperature $\neq 1$, center against the sampling law: $c_u = \log q(y_u \mid h_u) + H(q(\cdot \mid h_u))$.

Under truncation with centering against untruncated p_θ, q concentrates on high-probability tokens, so $\mathbb{E}_q[\log p_\theta] > -H(p_\theta)$ and $\mathbb{E}[c_u] > 0$ — a silent positive bias with no other visible symptom. Test V1 detects it.

1.10 Two required statements about $\hat{D}_m$ (it stays, so these must be correct)

(a) $\hat{D}_m$ requires $M \geq 3$. At $M{=}2$, $D_1 = d(1,2) = D_2$ by symmetry, so $\mathrm{sd}(D) = 0$ and $\hat{D} = 0/0$. The draft handles only $M{=}1$. Add to §2.3: *"$\hat{D}_m \equiv 0$ for $M \leq 2$, since disagreement is undefined for a single particle and degenerate for two."* Implement the guard (§2.4) before running any $M{=}2$, $\lambda_D > 0$ cell — Table 12 currently lists one.

(b) γ and λ_D are not on the same scale. $\bar{H}$ is in nats spanning roughly 0–4 (Figure 7 row 2), so $\gamma\bar{H}$ at $\gamma{=}0.3$ ranges over ~0–1.2. $\hat{D}$ is a z-score bounded by $\sqrt{M-1} \approx 1.73$ at $M{=}4$, so $\lambda_D\hat{D}$ at $\lambda_D{=}0.3$ ranges over ~ ± 0.52. Equal coefficients are not equal tilts. Either report both channels' realized spread, or calibrate λ_D so that $\mathrm{sd}_m(\lambda_D\hat{D}_m)$ matches $\mathrm{sd}_m(\gamma\bar{H}_m)$, and say which.

1.11 Delete the §3.1 attribution sentence

"Since both variants share the entire pipeline apart from the reliability term, the ensemble estimator—not the choice of penalty—is what produces this margin."

Every $\lambda_D > 0$ cell is currently $\boxed{-}$. Reinstate only once those rows exist, and only in the corrected form (§1.3, Correction 2): the two configurations differ in two coefficients, not one.

1.12 Table 6

Symbol	Meaning	Default
w	observation window	32
M, T	particles, particle length	4, 8
β	typicality coefficient (Eq. 3)	1.0
γ	entropy coefficient (Eq. 3)	set by the grid
λ_D	disagreement coefficient (Eq. 3)	0
λ_L	lookahead mixing (Eq. 9)	1.0
ϵ	renormalization guard (Eq. 7)	10^{-6}
κ, α	robust aggregation (App. C)	1, 0
τ; λ_k, λ_v	adaptive budgets; diversity	0; 0, 0

η and λ_H are removed; λ_D is unchanged. **Rename the robust-aggregation β (App. C) to κ** — it now collides with the typicality coefficient.

Part 2 — Code changes

2.1 Capture: per-step statistics from the sampling distribution

Replaces the per-particle logging in the capture path (Appendix E.1, step ii). The disagreement path is untouched.

```
python
```

```python
def step_stats(logits, y, temperature=1.0, top_k=None, top_p=None):
    """Per-step reliability statistics. logits: [M, |V|]; y: [M] sampled ids.
    Must be called on the SAME logits the sampler consumed, with the SAME
    truncation, or E[c|h] != 0 (paper §1.9). Returns logp, H, V, c — all [M]."""
    z = logits / temperature
    if top_k is not None: z = mask_top_k(z, top_k)    # -inf outside support
    if top_p is not None: z = mask_top_p(z, top_p)

    logq = torch.log_softmax(z, dim=-1)           # centering law == sampling law
    q    = logq.exp()

    # masked reductions: q == 0 with logq == -inf gives nan under plain q * logq
    qlogq  = torch.where(q > 0, q * logq,      torch.zeros_like(q))
    qlogq2 = torch.where(q > 0, q * logq.pow(2), torch.zeros_like(q))

    H  = -qlogq.sum(-1)                       # entropy
    V  = qlogq2.sum(-1) - H.pow(2)            # varentropy == Var[log q]
    lp = logq.gather(-1, y.unsqueeze(-1)).squeeze(-1)
    return lp, H, V, lp + H                   # c = lp + H
```

Store **per step**, not only trajectory means — tests V3 and V4 need the sequence. Cost is $M \times T$ floats per prompt.

2.2 Reliability weights

```python
def reliability_weights(c, H, V, D_hat, beta, gamma, lambda_D=0.0, normalize=False):
    """c, H, V: [M, T]. D_hat: [M], from the unchanged Eq. 4/5 path.
    Returns omega: [M]."""
    T = c.shape[-1]
    c_bar, H_bar, V_bar = c.mean(-1), H.mean(-1), V.mean(-1)
    x = c_bar / (V_bar / T).clamp_min(1e-8).sqrt() if normalize else c_bar
    R = beta * x - gamma * H_bar - lambda_D * D_hat
    return torch.softmax(R, dim=0)            # no eta (§1.7)
```

2.3 Guard $\hat{D}_m$ at small M (§1.10a)

```python
```

```
def standardize_disagreement(D):                # D: [M], from Eq. 4
  M = D.shape[0]
  if M <= 2:                                    # undefined at M=1, degenerate at M=2
    return torch.zeros_like(D)
  sd = D.std(unbiased=False)
  return torch.zeros_like(D) if sd < 1e-8 else (D - D.mean()) / sd
```

2.4 Config

```
diff
- lambda_H: float = 0.3
- eta:    float = 1.0
  lambda_D: float = 0.0       # unchanged; still the disagreement switch
+ beta:    float = 1.0
+ gamma:   float = 0.0
+ normalize: bool  = False     # ablated switch, not a default; see V5
```

To reproduce published runs: `beta=1.0, gamma=1+lambda_H_old, lambda_D=lambda_D_old, normalize=False`.

2.5 Analysis module

Keep `disagree()` and `standardize_disagreement()` on the scoring path (they are live at $\lambda_D > 0$). Additionally log raw D_m on **every** capture regardless of λ_D, as the draft already does, so test S4 runs on any capture. $\bar{c}_m$ averages over each particle's own steps, so it handles ragged trajectories natively. The truncate-to-shortest-trajectory rule (Appendix D.1) is still required for D_m and stays.

Part 3 — Experiments

3.0 What runs on existing captures

Appendix E.1 already stores per-particle average log-probability and mean predictive entropy separately, so

$$\bar{c}_m = (\text{avg logprob}_m) + (\text{mean entropy}_m)$$

is **exactly recoverable with no re-decoding**, over the 4,400 LongBench and 52,000 RULER particles of Figure 7. Tests V1, V2, D1, D2, S4 run today. Tests V3, V4, V5 need one re-capture pass (§2.1) to obtain V_u and per-step c_u — same MT decode, roughly 1 GPU-hour over 550 + 6,500 prompts.

3.1 Validation

ID	Computation	Expected	Failure means
V1	grand mean of $\bar{c}_m$ with bootstrap CI	CI contains 0	**positive ⇒ truncation mismatch (§1.9). Fix §2.1 and re-capture before anything else**
V2	$\|R^{\text{logged}} - (\bar{c} - (1+\lambda_H)\bar{H} - \lambda_D \hat{D})\|_\infty$	$< 10^{-5}$	code $\neq$ Eq. 3; an exact T-fold gap confirms §1.8
V3	lag-k autocorrelation of $\{c_u\}_{u=1}^{T}$	≈ 0 for all k	non-zero ⇒ truncation, or a repetition regime; drop the $\tilde{c}$ normalization
V4	OLS of per-prompt $\widehat{\text{Var}}_m(\bar{c}_m)$ on $\frac{1}{T^2}\sum_u \bar{V}_u$	slope ≈ 1	Prop. 3 miscalibrated; report unnormalized only
V5	$s_X = \sqrt{\bar{V}_X / T}$; ratio q_{95}/q_{05} across prompts	—	< 1.3: keep `normalize=False`, demote Prop. 3 to a remark. > 2: normalization is load-bearing, promote it

python

```
# V1  (existing captures, no GPU)
c_bar = cap["logp_mean"] + cap["H_mean"]              # [N_particles]
print(c_bar.mean().item(), bootstrap_ci(c_bar, 10_000))

# V2
rhs = c_bar - (1 + LAMBDA_H) * cap["H_mean"] - LAMBDA_D * cap["D_hat"]
assert torch.allclose(cap["R_logged"], rhs, atol=1e-5), (cap["R_logged"] - rhs).abs().max()

# V3  (needs per-step c from §2.1)
for k in range(1, T):
    print(k, pearson(c[:, :-k].ravel(), c[:, k:].ravel()))

# V4
emp  = c_bar.view(P, M).var(dim=1, unbiased=True)      # per prompt
pred = V.view(P, M, T).mean((1, 2)) / T
print(ols_slope_r2(pred, emp))

# V5
s = (V.view(P, M, T).mean(-1) / T).sqrt().mean(-1)
print((s.quantile(0.95) / s.quantile(0.05)).item())
```

3.2 Decomposition diagnostic — motivates §1.1–1.3, no GPU

Within each prompt, across its M particles, decompose the across-particle variance of the log-probability term $L_m = \bar{c}_m - \bar{H}_m$. The softmax sees only this spread:

$$\mathrm{Var}_m(L) = \mathrm{Var}_m(\bar{c}) + \mathrm{Var}_m(\bar{H}) - 2\,\mathrm{Cov}_m(\bar{c}, \bar{H}).$$

```python
cb, Hb = c_bar.view(P, M), cap["H_mean"].view(P, M)
L      = cb - Hb
var_L, var_c, var_H = L.var(1), cb.var(1), Hb.var(1)
cov    = ((cb - cb.mean(1, keepdim=True)) * (Hb - Hb.mean(1, keepdim=True))).sum(1) / (M - 1)
shares = torch.stack([var_c, var_H, -2 * cov], -1) / var_L.unsqueeze(-1)   # D1
rho    = spearman_within_prompt(cb, Hb)                                    # D2
```

Reading D1. If $\mathrm{Var}_m(\bar{H})$ dominates, the draft's "log-probability weighting" is operationally an entropy penalty, and separating the channels is necessary rather than cosmetic. If $\mathrm{Var}_m(\bar{c})$ dominates, the confound is mild — report that honestly and keep the change for interpretability. Report both shares and the within-prompt correlation, per benchmark.

3.3 Score-level sweep — re-weights captured particles, no decoding

ID	Metric	Reuses
S1	coverage of $\mathcal{Q}_{\text{future}}$ (Eq. 17) per $(\beta, \gamma, \lambda_D)$, nested prefixes $M \in \{1, 2, 4, 8\}$	Figure 4
S2	oracle $\mathrm{Recall@}B$ (Eq. 18) per $(\beta, \gamma, \lambda_D)$	Figure 6
S3	marginal Spearman ρ of each signal vs particle utility — add $\bar{c}_m$ and $\tilde{c}_m$ to the existing list ($\log p_\theta, -\bar{H}_m, -\hat{D}_m, R_m$)	Figure 7 row 3
S4	**incremental** value of disagreement: semipartial ρ of D_m with the residual of utility on $(\bar{c}_m, \bar{H}_m)$; ΔR^2; LRT against a within-prompt permutation null (1,000 draws)	Figure 7 capture
S5	top-B overlap and Spearman ρ between retained sets under $(0, 0, 0)$, $(1, 0, 0)$, $(1, 1, 0)$, $(1, 1.3, 0)$	Figure 5

S3 is the key result for the reparametrization: if $\bar{c}_m$ outranks raw $\log p_\theta$ on both benchmarks, removing the confound demonstrably improves the per-particle signal — measured on the paper's own existing diagnostic. **S4 is the gate for $\lambda_D > 0$.** Run it per benchmark; Figure 7 already shows $-\hat{D}_m$ at $\rho = -0.26$ on LongBench and $+0.03$ on RULER, so a pooled number would hide a sign flip. A non-zero, sign-consistent semipartial justifies carrying $\lambda_D > 0$ into the end-to-end grid; a near-zero one is a reportable negative control — disagreement subsumed by the model's own conditional uncertainty structure — and λ_D stays at 0 with the switch documented.

S5 gates §3.4 entirely. If overlap > 0.98 across all four rules, no end-to-end grid can resolve them; report score-level only and say so.

3.4 End-to-end grid

Stage 1 — the (β, γ) plane at $\lambda_D = 0$, unnormalized so the published cells relabel exactly:

$(\beta, \gamma, \lambda_D)$	Role	Status
$(0, 0, 0)$	uniform $\omega_m = 1/M$ — the missing control	run
$(1, 0, 0)$	pure typicality, entropy off	run
$(1, 1, 0)$	current "log-probability only"	✅ 40.29 / 41.39 / 41.30 / 41.40
$(1, 1.3, 0)$	current core	✅ 40.28 / 41.38 / 41.84 / 42.21
$(0, 1, 0)$	pure entropy	only if D1 shows $\bar{H}$ dominating
$(1, 2, 0), (2, 0, 0), (0.5, 0, 0)$	range probes	only if S1/S2 separate them

Stage 2 — the λ_D probe, gated on S4. At the best $(\beta^\star, \gamma^\star)$ from Stage 1, run $\lambda_D \in \{0, 0.3\}$ calibrated per §1.10b. Also run $(1, 0, 0.3)$ — disagreement *instead of* entropy, the configuration the `/D` variant was meant to be and could not previously express.

Protocol. $B = 128$ only for the grid: Table 3 shows the criterion separating by 5.05 points at $B{=}128$ and 0.42 at $B{=}1024$. Nested prefixes of a single 8-particle draw per prompt, so every cell re-weights identical particles and no cell differs by test-time compute. **Three seeds minimum** — the gaps being resolved are 0.01 (Table 4) and 0.04 (Table 11), and the existing M-sweep is non-monotone ($M{=}2$ at 41.10 / 41.22 sits below $M{=}1$ at 41.56 / 42.03), so the noise floor is unknown and may exceed them. If it does, "no weighting rule separates from uniform" is the correct and reportable result.